

What I Learned: Building My First AI API

A simple, step-by-step explanation of what I built and why

1. The Problem Statement

At the start of Week 1, the task was this:

"Build something where a person can type a question, send it to an AI model, and get back a clean, trustworthy answer — not just a chatbot reply, but something a real app could depend on."

That's harder than it sounds. Anyone can paste a question into ChatGPT and get an answer. But apps can't just "paste into ChatGPT" — they need something that behaves the same way every single time: predictable format, predictable cost, and a plan for when something goes wrong.

Think of it like this: Asking ChatGPT a question yourself is like asking a friend for directions — you get an answer, but it might be long, short, or in a different format every time. Building an API is like creating a GPS app: every single person who uses it must get an answer in the exact same format, every single time, no matter who's asking.

So the real assignment was: build a small web service — a POST /ask endpoint — that takes in a question and always replies with a clean, structured, checked answer, and put it live on the internet so anyone (with permission) could use it.

2. The Building Blocks — Explained Simply

What is an API?

API stands for "Application Programming Interface." It's just a way for two computer programs to talk to each other using a fixed set of rules, kind of like a menu at a restaurant.

Think of it like this: You (the customer) don't walk into the kitchen and cook your own food. You look at a menu, pick something, tell the waiter, and the kitchen sends back a plate of food. The API is the waiter and the menu combined — it defines exactly what you're allowed to ask for, and exactly what shape the response will come back in.

GET vs POST — and why I used POST

When your browser or app talks to a server, it uses different "verbs" to describe what kind of action it wants. The two most common are GET and POST, and they mean very different things:

	What it means
GET	"Just give me some information, I'm not sending you anything to

	process." Used for things like checking if the server is alive (/health) or loading a webpage.
POST	"Here is some data — please do something with it and send me back a result." Used when you're submitting something, like a form, a question, or a file.

Think of it like this: GET is like looking at a restaurant's menu on a poster outside — you're just reading information, nothing changes. POST is like placing an actual order — you're handing something over (your question) and expecting the kitchen to do work and hand something back (the answer).

In this project, I used:

- GET /health — just to check "is the server awake?" No data needed, just a yes/no.
- POST /ask — to actually send my question to the AI model and get back a real, structured answer.

That's exactly why /ask had to be POST: I was sending data (my question) and expecting the server to process it and return something new — not just fetch something that already existed.

What is JSON, and why "structured output"?

JSON (JavaScript Object Notation) is just a simple, universal way to organize data using labels and values, like a labeled form instead of a paragraph of free text. For example:

```
{ "answer": "Paris", "confidence": 0.95 }
```

Instead of letting the AI reply however it wants ("Well, I think it's probably Paris, but I'm not 100% sure..."), I forced it to always reply in that exact labeled format. That way, my code (or anyone else's app) always knows exactly where to find the answer, without having to guess or read paragraphs.

Think of it like this: It's the difference between asking a friend "how was the test?" and getting a whole rambling story, versus handing them a form that says Score: ___ / 100, Passed: Yes/No. The form version is boring, but a computer can actually use it reliably.

What are tokens, and why track cost?

AI models don't charge per question — they charge based on "tokens," which are small chunks of text (roughly a word or part of a word). Every question you send and every answer you get back uses up tokens, and each token costs a tiny fraction of a cent.

My endpoint returns tokens_used and cost_usd with every single answer, so I can actually see, in real numbers, what each question costs — instead of finding out only when a bill arrives.

What are guardrails, and why retry?

AI models are usually right, but not always — sometimes they reply in the wrong format, or leave something out. A "guardrail" is a checkpoint that catches this before it reaches the user.

My API works like this: it asks the model for an answer, checks if the response actually matches the expected format, and if it doesn't, it quietly retries once automatically. Only if it still fails does it report a clean error — it never just passes broken data along and hopes for the best.

***Think of it like this:** It's like a teacher checking a homework submission before accepting it. If a student turns in the wrong file format, the teacher doesn't just accept it and hope for the best — they ask the student to resubmit it properly first.*

3. What I Actually Did — Step by Step

Step 1: Got the project files

I downloaded (cloned) the starter project from GitHub onto my computer. This gave me all the base code I needed — the server code, a demo webpage, and instructions — without having to write everything completely from scratch.

Step 2: Created a clean workspace for the project

I set up something called a "virtual environment" — basically a private, isolated toolbox just for this project, so its tools and libraries don't mix up with anything else on my computer.

Step 3: Installed the tools the project needed

Using one command, I installed all the Python libraries the project depends on — things like FastAPI (to build the web server) and the OpenAI library (to talk to the AI model).

Step 4: Added my private API key safely

To let my code talk to OpenAI's models, I needed a secret "API key" — like a personal password. I stored it in a special file called `.env`, which is automatically excluded from anything I upload publicly, so my secret key never gets exposed.

Step 5: Started my API server

I ran a command that started my FastAPI server on my own computer, listening for requests at a local address. This is the "engine" that actually receives questions and returns answers.

Step 6: Started the demo webpage

In a second terminal, I started a simple Streamlit webpage — a visual, clickable interface — so I could test my API by typing a question and pressing a button, instead of typing complicated commands every time.

Step 7: Asked real questions and checked the answers

I typed in real questions and watched the responses come back as clean, structured JSON — including the answer itself, which model replied, how many tokens it used, how long it took, and how much it cost.

Step 8: Tested the guardrail (validation + retry) system

I turned on a special test mode that forces the model to reply badly on purpose, just to see my code catch the mistake, log it, and automatically retry — proving the safety net actually works before trusting it in real use.

Step 9: Checked that my secret key was safe

Before saving my code anywhere public, I double-checked that my .env file (holding my private key) was never included — confirmed my project was clean and safe to share.

Step 10: Saved my project to my own GitHub account

I created my own personal repository (an online storage space for code) on GitHub, and uploaded ("pushed") my project there. This is what let me connect it to a hosting service afterward.

Step 11: Deployed it live on the internet

I connected my GitHub repository to Render (a free hosting service), told it exactly how to start my server, and gave it my secret key securely (without ever putting it in the code itself). Within a couple of minutes, my project was live with its own public web address.

Step 12: Verified it was really working, live

I opened my project's live web address and checked its auto-generated documentation page, which lists exactly what my API can do. Then I sent a real test question straight to the live version and got back a proper, structured answer — proving it wasn't just working on my laptop, but for real, on the internet.

4. What I Learned, In One Paragraph

I learned that there's a big difference between "getting an AI to answer a question" and "building something reliable that other software can depend on." A reliable system needs a clear contract (structured JSON), a safety net for when the AI misbehaves (guardrails with retries), visibility into what everything costs (tokens and dollars), and a real, live home on the internet (deployment) — instead of just living on my own laptop.

My Live Project

Public API URL: <https://anirudh-ai-internship.onrender.com>

GET /health → checks the server is alive. GET /docs → shows the full interactive documentation. POST /ask → send a question, get back a structured, validated answer.